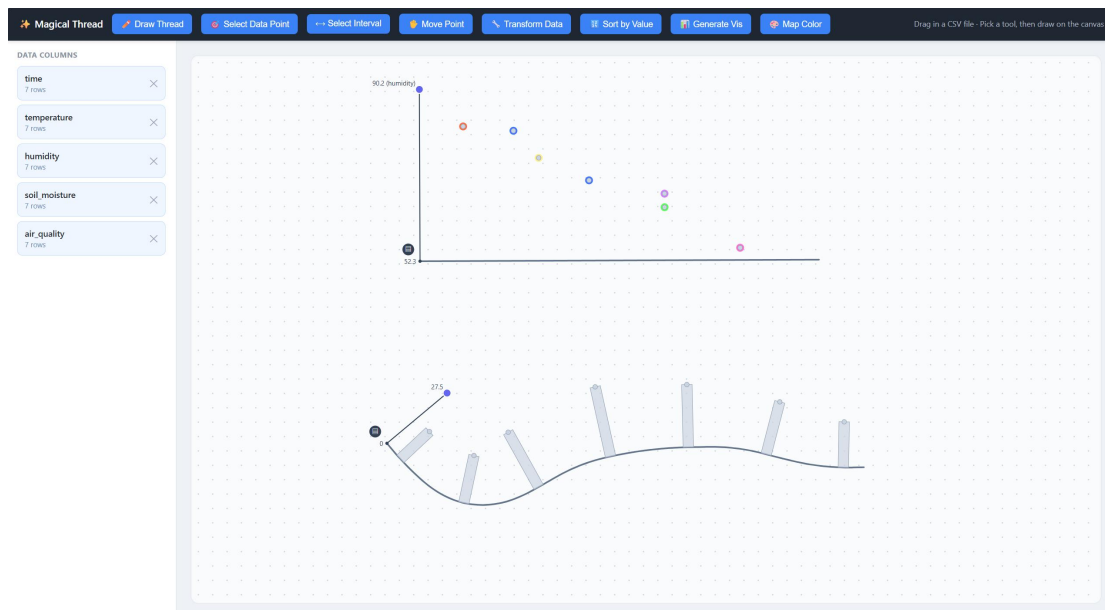


Magical Thread Report

(draft example, not final)

1. Abstract

Most data visualization tools separate authoring from exploration: users select a chart type from a fixed gallery, bind data to predetermined encodings, and obtain an opaque result that is difficult to question or reshape. This makes open-ended exploration of one's own data awkward, since altering the view often means starting over. We present Magical Thread, a canvas-based visualization tool in which users freely construct visualizations by hand-drawing a thread, dragging data onto it, and stacking composable function cards that transform, encode, and re-express the data. Rather than choosing a chart, users build one incrementally: scatter, line, bar, and multi-dimensional plots all emerge from the same lightweight primitives. Each function card is a concrete, draggable object that can be enabled, disabled, deleted, reordered, and copied, forming an ordered, order-sensitive, and fully non-destructive pipeline in which every automated suggestion remains a proposal the user can override. Central to our design is the treatment of the thread as an interaction substrate: a single, minimal primitive that carries data, geometry, and operations, and onto which visualizations are composed. A key question we investigate is the expressiveness of this substrate, specifically how wide a range of visualizations and exploratory actions can be supported by threads and function cards without introducing new, special-purpose machinery. We describe the system's design and architecture, in which each new capability is added purely by registering a new card type, and through a set of use cases we show that treating the thread-plus-pipeline as the primary artifact makes visualization construction itself a transparent, reversible, and extensible medium for data exploration.



The main interface with generated visualizations

2. Introduction

Data visualization is not only a means of presenting findings but also a means of finding them. When people work with their own data, they rarely know in advance what view will be revealing; instead, they build a chart, notice something, adjust it, try a different encoding, and gradually converge on an understanding. This open-ended and iterative process is the essence of exploratory data analysis. Yet the tools people most commonly reach for are poorly suited to it.

Mainstream visualization tools tend to treat authoring as a discrete, up-front decision. Spreadsheet and template-based tools ask the user to pick a chart type from a fixed gallery and bind data to predetermined slots, producing a finished artifact whose internal logic is opaque. Grammar- or specification-based tools (e.g., those in the Vega/ggplot lineage) are far more expressive, but they demand that the user commit an entire visualization to a formal specification, which is difficult to author incrementally and offers little in the way of direct, in-place manipulation. In both families, changing one's mind is expensive: adjusting the view often means reselecting a template, rewriting a specification, or starting over. The process of exploration consists of the sequence of small, reversible decisions that leads to insight is discarded rather than made into something the user can hold onto, inspect, and reshape.

We argue that a better fit for exploration is a tool in which visualizations are constructed, not selected, and in which the construction process is itself a first-class, manipulable object. We present Magical Thread, a canvas-based visualization tool built around a single unifying primitive: the thread, a freeform curve that the user hand-draws on the canvas. The thread is designed as an interaction substrate, a minimal, general-purpose carrier onto which everything else is composed. Data is dragged onto the thread and distributed along its length; operations are attached to it as an ordered stack of function cards; and the visible visualization is simply the outcome of running these cards in sequence. All of this is progressive and reversible: any card can be disabled, deleted, reordered, or copied, and the pipeline re-runs to reflect the change, so nothing is ever destroyed. Rather than offering "scatter plot" or "bar chart" as monolithic choices, Magical Thread lets these views emerge from the same small set of composable primitives - binding data along the curve, selecting and transforming points, projecting additional dimensions, and mapping values to position or color.

By committing to the thread as a substrate, we raise a central research question: how expressive can such a minimal substrate be? That is, how wide a range of visualizations and exploratory actions can be supported by threads and function cards alone, without falling back on new, special-purpose mechanisms for each chart type? Much of the design of Magical Thread is an attempt to answer this question by construction, showing that scatter plots, line and bar charts, multi-dimensional plots, and color-encoded views can all be reached as compositions over the same substrate. Where a new capability is needed, it is introduced not as a bespoke feature but as a new kind of card that reuses the substrate's existing structure, and we use this discipline as a probe: the more that new visualizations reduce to new cards, the more expressive the substrate has proven to be.

This design is also a deliberate response to the shortcomings above. Because every operation is reified into a concrete card, the user can see exactly what their visualization is made of and take it apart at any point. Because the cards form an order-sensitive, non-destructive pipeline, exploration becomes a matter of rearranging and toggling cards rather than committing to a specification. And because the system's automated behaviors (such as smoothing a rough curve or suggesting a shape) are themselves expressed as cards, every piece of "magic" remains a proposal the user can override simply by disabling the corresponding card which preserves the user's sense of control even as the tool offers help. Architecturally, this same commitment pays off in extensibility: a new capability is added purely by registering a new card type, without modifying any existing behavior, so the vocabulary of exploration can grow over time.

This paper makes the following contributions:

The thread as an interaction substrate for exploratory visualization, a single minimal primitive that carries data, geometry, and operations, onto which a wide range of visualizations are composed rather than selected.

A study of the expressiveness of this substrate, demonstrating by construction how scatter, line, bar, multi-dimensional, and color-encoded visualizations all emerge from threads and composable function cards without special-purpose machinery.

The reification of visualization operations as first-class, manipulable function cards, forming an order-sensitive and fully non-destructive pipeline that makes the exploration process transparent, reversible, and controllable.

3. Design

Two commitments shape Magical Thread. The first is that a visualization should be constructed rather than selected: the user starts from a stroke and arrives at a scatter, a bar chart, or a two-dimensional encoding through a sequence of small local actions, never having to name in advance what they are building. The second is that the construction process itself should remain on the canvas as a manipulable artifact. In exploratory work the sequence of decisions like which points to select, which values to fold or in what order carries as much meaning as the final image, and a tool that collapses it into an opaque picture and a linear undo stack makes changing one's mind expensive. Everything else in the design follows from these two, including our treatment of the system's automated behaviors, which are never allowed to be exceptions to the second commitment.

3.1 The Thread as substrate

The system's only user-created primitive is the thread: a free form curve drawn by hand. We intend it as an interaction substrate in Beaudouin-Lafon's sense, a medium that holds content and supports operations upon it, rather than a widget with a fixed repertoire of behaviors. The thread is deliberately impoverished: it has no axes, no scales, no chart identity. What it offers is an ordered, embedded, one-dimensional space, and this proves sufficient to carry the three things a visualization needs.

It carries geometry, because a curve parameterized by arc length gives every mark a position expressible independently of the curve's actual shape; a straight thread and a circular thread are the same substrate seen through different geometries, and a linear layout, a curved one, and a radial one are the same construction rather than three chart types. It carries data, because dragging a column onto a thread distributes its rows along the curve, making the thread a carrier of a data sequence in which row order is position, it is a relation rather than a transfer, so the same data may live on several threads. And it carries operations, in the form of the ordered pipeline described below, which is what turns a drawn stroke into a computational object.

Because the thread is the only thing the user creates directly, all further interaction reduces to acting on or with threads. This is what makes our research question tractable: instead of asking whether the system supports chart type X, we ask whether X is expressible as a configuration of threads and operations over them.

3.2 Operations reified as function cards

Every operation applied to a thread is reified into a concrete object which is a function card. Drawing, shape correction, binding a column, selecting, transforming, sorting, encoding color: none is a transient command; each is a persistent object attached to the thread. The card is the operation. This extends to visualization operations the move that object-oriented drawing makes for attributes, pulling properties out of modal dialogs and giving them independent existence so they can be inspected, shared, and reused.

The cards on a thread form an ordered pipeline, and the visible visualization is defined at all times as the result of running the enabled cards in sequence. Nothing is stored as a finished picture. A card can therefore be disabled, deleted, reordered, or dragged onto another thread, where it applies the same analytical step to different data. Reification thus yields both polymorphism and reuse: a card is defined over the substrate's contract of geometry, rows, and selection rather than over a chart type, so selection means the same thing on a straight thread of five points and a circular one of five hundred; and an operation the user has discovered becomes something they can carry.

Order matters, and we treat that as a feature rather than an inconvenience. Selecting then sorting is a different analytical act from sorting then selecting, and the pipeline preserves the difference instead of normalizing it away. Since order is expressed spatially, exploring that difference is a matter of dragging a card. We never silently reorder the user's stack to produce a result the system considers more sensible; where an operation's correctness would otherwise depend on its position, we design it to be position-independent instead.

Non-destructiveness is what makes this safe. No card mutates what it receives; each contributes a layer of interpretation, so disabling one removes exactly its contribution and the view returns precisely to what it would have been. This is also how we handle the system's inferences: recognizing that a rough stroke was meant to be a line or a circle, smoothing a shaky curve, detecting that a column encodes time. Each appears as an ordinary card, indistinguishable in status from one the user created. The "magic" we want, the sense that the tool understood what you meant, therefore costs the user no authority: the tool's interpretation sits in the same space and under the same controls as the user's own decisions, and overriding it is not an escape hatch but the ordinary act of switching off a card.

3.3 Instruments and the reuse of gestures

Threads and charts are domain objects; tools, including drawing, selecting, moving a point, and transforming, are instruments in the sense of instrumental interaction, mediators the user picks up and applies to whatever object they choose rather than behaviors owned by particular objects. Function cards sit between the two: a card is the trace an instrument leaves on an object, reified into an object in its own right. The consequence is that the interaction vocabulary does not grow with the number of representations; one selection instrument works on every thread and every chart, however that chart was built.

We push the same economy into gestures. A single physical action, such as picking up an object and dropping it on a target, carries a meaning determined entirely by what it connects. Dropping a data column on a thread binds data to geometry; dropping a thread on a chart's axis contributes a spatial dimension and promotes a one-dimensional layout to a two-dimensional one; dropping a thread on an encoding slot contributes a visual channel instead; dropping one mark on another re-expresses the relationship between marks, connecting them into a continuous form or grounding them against a baseline; dropping a card on another stack transfers an operation. In the spirit of Data Illustrator, the user expresses structural intent by arranging things spatially and the system infers the corresponding change. One practical effect is that changing representation requires no chart-type menu at all: a line or a bar chart is the consequence of a relationship the user drew between two marks.

3.4 Separating the data substrate from the visualization

An earlier version of the design attached visualization operations directly to the thread, treating it as both data carrier and picture. This proved limiting, since a visualization is often about several threads and a thread is often the source of several visualizations; binding them one-to-one forced the user to destroy one view in order to build another. We therefore separate the thread, which carries what the data is and where it lies, from the chart, a distinct

object that carries how the data is shown. A chart is created beside its source rather than replacing it, so the user never loses the structure they built while looking at a view of it.

A chart captures the pipeline that produced it as its own editable material, which lets the user revise the analysis within a view by changing a selection or adjusting a transformation, without disturbing the thread it came from, and so keep one stable working structure behind several divergent views. Whether a given source is mirrored, so that later changes to the thread flow into the chart, or held as a detached copy, is a choice the user makes per source rather than globally: mirroring is what one wants when a chart is a live window onto ongoing work, detachment when it is a preserved alternative, and a single chart may reasonably be live in one dimension and frozen in another.

This separation also gives a principled account of encoding. Once a chart can draw on several threads, a dimension is simply a thread contributed to a slot, and the slot decides the channel, which may be position along the curve, offset from it, or color. A new visual channel is thus not new machinery but a new kind of slot over the same substrate.

We hold to that discipline throughout, and treat it as more than engineering hygiene. Every capability in the system was introduced additively, as a new card, instrument, or slot defined against a contract that may be extended but never redefined, and none required altering an existing behavior. This is how we operationalize our research question: each time a new visualization reduces to a new card over the same substrate, the substrate has demonstrated reach, and each time it would demand a special-purpose mechanism, we have located a boundary.

4. System

4.1 The Canvas

Magical Thread presents a single unbounded canvas, a slim toolbar of instruments, and a sidebar holding the data currently in play. There is no chart gallery, no property inspector, and no modal configuration; the canvas is the whole of the workspace, and everything the user makes, including threads, charts, and the cards attached to them, lives there as an object that can be moved, opened, and taken apart.

Data enters by dragging a CSV file onto the canvas. The system parses it and places each column in the sidebar as a small card, the only place in the system where data appears in tabular rather than spatial form. Columns remain there as reusable material: a column is never consumed by being used, and the same column can be dropped onto as many threads as the user likes. The system notices when a column appears to encode time and keeps that association with the column so that later views can surface it, but this is a background observation rather than a commitment. Nothing about the layout depends on it until the user makes it so.

A thread is drawn with the drawing instrument in one stroke. Once drawn it behaves like any other object on the canvas: with no instrument active, the user can grab the curve anywhere and move the whole thread, along with everything bound to it. The movement is recorded as part of the thread's construction rather than applied to it, which is why it can be undone like any other step.

Each thread carries a small badge at its far end showing how many cards it holds. Clicking the badge opens the thread's pipeline as a panel beside it, which contains a vertical stack of cards in execution order. Each card can be switched off, removed, dragged to a new position, or dragged into another thread's panel to be copied there. The panel is the only "interface" in the

system beyond the toolbar, and it exists to make the thread's construction legible in place rather than in a separate pane.

4.2 From a stroke to a plot

The typical session begins with a stroke and ends, several small steps later, with a chart, without the user ever having chosen one.

Drawing and correction. A freehand stroke is rough. The moment the stroke is completed, the system examines it and decides whether the user was drawing a straight line, a circle, or something else, and appends a correction card that regularizes the curve accordingly. It may snap the curve to a line, close it into a circle, or, failing a confident match, smooth the tremor out of it. The card names what the system concluded, so the interpretation is visible rather than merely applied, and switching it off restores the original stroke exactly. This is the system's first and most immediate piece of "magic," and it sets the pattern for all the others.

Binding. Dragging a column from the sidebar onto a thread distributes its rows along the curve in order, with a small margin at each end so that the endpoints of a closed curve do not collide. The thread now shows a run of marks; hovering one reveals its value, and its time if the column had one. From here the shape of the thread is no longer decorative. It is the layout.

Selecting. Two instruments make selections: one picks individual points, the other picks an interval by clicking its two endpoints. Both are additive, so a selection can be built up across several gestures, and both record their result as a card, which means a selection is a persistent part of the construction rather than a transient highlight. Selected points are drawn dark and remain so across later operations.

Adjusting and transforming. A point can be dragged along the curve to a new position without disturbing the others, which lets the user pull apart a crowded region or foreground a particular value. More substantially, the transform instrument applied to a selection offers two families of operation. Fold collapses the selected points into a single point that stands for their aggregate, which may be the mean, sum, extremum, median, count, range, spread, or endpoints. The point is positioned at the middle of the region it replaces and drawn as a triangle so that its derived status is visible. Normalize leaves the points in place and rewrites their values, rescaling to a unit range, standardizing, taking logarithms or roots, or expressing each as a share of the total; these are drawn as squares. Each transformation produces its own card, so a sequence of them can be unwound one at a time, and the resulting points are ordinary points in every other respect. They can be hovered, selected, moved, and transformed again.

Sorting. Sorting is applied by activating the instrument and clicking a thread, which reorders its points by value along the curve. Because it is a card like any other, its position relative to a selection card is meaningful and adjustable: selecting the first five points and then sorting is visibly different from sorting and then selecting the first five, and the user can see and change which they meant.

4.3 Charts

At any point the user can turn a thread into a chart. The generate instrument, applied to a thread, creates a chart beside it rather than in place of it: the thread stays where it is, still editable, while the chart takes a copy of its construction and becomes an independent object with its own position on the canvas.

The chart draws the same curve as a baseline and lifts each point off it in proportion to its value, perpendicular to the curve at that point. Because the offset follows the local normal,

this works on any shape. A straight thread yields a conventional scatter plot, a curved one yields a plot that follows the curve, and a circular one yields a radial plot. None of these is a separate mode. A short axis stands at the start of the curve showing the value range; dragging its handle changes the scale, and every point moves with it. The chart carries its own badge and its own panel, which shows the operations it inherited from each source thread alongside the visual operations that belong to the chart itself.

Changing the representation is done by dragging on the chart rather than by choosing from a menu. Pressing on a point and pulling produces a dashed tether; releasing it on another point connects the marks into a line chart, while releasing it on the point's own base on the curve grounds the marks into bars. Both are recorded as a card on the chart, so the chart can be returned to a plain scatter by switching it off. Bars are drawn along the local normal and narrow automatically as the number of points grows.

Adding a dimension reuses the gesture that moves objects. Dragging one thread onto a chart's axis causes the axis to highlight as the thread approaches it and contributes that thread's data as a second dimension. The chart then shows the relationship between the two dimensions. The source thread is not consumed and remains fully visible on the canvas. Because the second dimension is resolved against the chart's final points rather than assumed to align with them, a folded point on one axis correctly aggregates the corresponding values on the other, and the result is the same whether the fold was applied before or after the dimension was added. The user is never required to arrange their cards in a particular order to get a correct answer.

Selecting on a chart works as it does on a thread, but writes into the chart's own copy of the construction. Clicking a point in a detached chart therefore changes that chart's selection alone, leaving the source thread and any sibling charts untouched.

4.4 Color

Color is treated as another dimension rather than a formatting option, and it is specified by drawing rather than by picking. Applying the color instrument to a thread opens a color disk on which the user lays out a curve of their own: a spline whose endpoints correspond to the extremes of the bound data and whose shape determines the path the colors take between them. Points can be added along the spline and dragged anywhere on the disk, and each takes the color of the position it occupies, so a mapping is authored as a gesture through color space rather than assembled from a list of stops. Values between control points are read off the spline by interpolating position, which means the user sees the entire ramp they are creating rather than the endpoints of it. The mapping is stored as a card on the thread and reopening it is a matter of clicking that card.

A thread carrying a color mapping can then be dropped into a chart's color slot, which appears in a small panel beside a mark when the user clicks it with no selection instrument active. The chart takes that thread on as a further source and colors each mark by its value under the mapping. As with spatial dimensions, the user chooses whether the color source is mirrored, so that editing the ramp on the original thread updates the chart live, or detached, so the chart's coloring can be tuned without touching the thread it came from. Removing the binding returns the marks to their default appearance.



The color mapping panel

5. Limitations

The substrate is one-dimensional. A thread supplies an ordered position space and a direction normal to it, which together account for the two spatial channels our charts use.

Representations that do not decompose into a path and an offset from it fall outside what the substrate can express. Matrix and grid layouts, treemaps, node link diagrams, and geographic projections all assume a two-dimensional space that is not derived from a curve, and building them as threads would amount to using the substrate to simulate something it is not. Scale is a related boundary. Because every row is placed individually along the curve and remains individually selectable and transformable, the system is comfortable with tens or hundreds of points and not with tens of thousands, where the mark ceases to be the right unit of interaction. We regard these as genuine limits of the substrate rather than missing features, and they are the most informative result of our additive discipline: the cases we did not build are precisely the cases that would have required special-purpose machinery.

Reification does not scale with the analysis. Cards make a construction legible, but a thread that has accumulated a long sequence of operations presents a long stack, which becomes harder to read exactly as the analysis becomes interesting enough to want to read. We offer no grouping, naming, or summarization of cards, and no way to save a subsequence as a reusable compound operation. A single card can be carried to another thread, but a workflow cannot. The order-sensitivity we treat as expressive is a burden in the same way: the user owns the meaning of their ordering, and the system gives no account of why a given arrangement produces the result it does.

6. Future Work

The most immediate direction is to widen the substrate's vocabulary of channels. Our treatment of color established the general form: a dimension is a thread contributed to a slot, and the slot decides how that thread's values are expressed. Size is the obvious next slot, mapping values to mark area to obtain bubble-style plots, and shape, opacity, and stroke weight follow the same pattern. The same would apply to channels that affect the thread itself rather than its marks, such as varying a line's thickness or texture along its length. Each of these is an addition rather than a mechanism, which is the point: if the substrate is as expressive as we claim, its channel vocabulary should grow without the existing ones being touched. Authoring such mappings by drawing, as we do with the color disk, also generalizes

in an interesting way, since a size or opacity ramp can equally be specified as a curve the user shapes rather than a formula they write.

A second direction is to make composition scale. Threads can already contribute dimensions to one another's charts, but a canvas of several interrelated views currently offers no visible account of its own dependency structure, and a long pipeline offers no way to name, group, or collapse the reasoning it records. We would like cards to be summarizable and, more importantly, reusable in groups. This would allow a sequence of operations to be saved as a compound card and applied elsewhere, so that a discovered method, and not merely a discovered step, becomes something the user can carry. The same reasoning applies across users: an analysis expressed as a stack of objects is, in principle, something that can be shared, read, and criticized by someone other than its author.

Third, the boundaries identified above invite work on the substrate itself rather than on what is built over it. Aggregation is the natural response to scale: if a thread cannot usefully hold tens of thousands of marks, it might hold binned or sampled marks that can be opened up in place, keeping the per-mark interaction model intact at a coarser grain. Relationships between threads are a further possibility. Threads that intersect, branch, or nest would bring hierarchical and connected structures within reach without abandoning the curve as the underlying medium.

Finally, we intend to evaluate the system with users, both to test whether the reversibility and transparency we designed for actually change exploratory behavior, and to observe where people reach for a representation the substrate cannot give them.